DEPARTMENT OF SOFTWARE ENGINEERING

VERITAS UNIVERSITY ABUJA

VUA SEN305 Practical Note

Lab Session 3 Domain Modelling MySQL ORM and Migrations

Semester Project

Enterprise Service Request and Maintenance Platform

Prepared by Bonaventure Anselm

Practical Session Overview

This session converts the public Django site built in Lab Session 2 into a database-backed enterprise application. You will install and configure MySQL, model the first business entities, generate and inspect migrations, apply database constraints, register models in Django Admin, create related records, query them through the ORM, and prove that invalid data is rejected.

Item	Details
Course	VUA-SEN305 Web Application Development Practical
Lab session	3
Duration	4 hours guided laboratory work
Starting point	Completed Lab Session 2 feature branch merged into develop
Project increment	Persistent service-request domain model
Backend	Python 3.12 and Django 5.2
Database	MySQL 8 with utf8mb4
Main evidence	ERD, migrations, admin records, ORM queries and integrity tests

Practical Outcomes

- Create a dedicated MySQL database and least-privilege application user.
- Move Django configuration from SQLite to MySQL using environment variables.
- Translate a business workflow into related Django models.
- Generate, inspect and apply migrations without editing the database manually.
- Enforce uniqueness, referential integrity, valid priority values and workflow timestamps.
- Use Django Admin to create and inspect related enterprise records.
- Use the ORM shell for create, filter, update, aggregate and relationship queries.
- Prove that invalid data and protected deletions fail as designed.
- Commit a clean, reproducible database increment to Git.

Domain Increment for This Session

Model	Purpose	Important relationship
Category	Classifies requests such as Electrical or ICT	One category has many service requests
ServiceRequest	Stores the reported issue and its workflow state	Belongs to requester and category
WorkOrder	Stores technician assignment and work progress	One service request has zero or one work order
Comment	Stores discussion and progress notes	Many comments belong to one service request

Laboratory Rules

- Use a local development database only. Do not use a production server or real personal data.

- Keep passwords in backend/.env and keep that file out of Git.
- Run python manage.py check after every configuration or model change.
- Read every generated migration before applying it.
- Never repair migration problems by deleting random migration files or editing MySQL tables manually.
- Capture the exact error message before correcting any failed command.

Part A Confirm the Lab Session 2 Starting Point

Step A1 Open the repository and activate the environment

Terminal

```
cd sen305-service-platform
git status
git switch develop
git pull
git switch -c feature/mysql-domain-model

# Windows PowerShell
.venv\Scripts\Activate.ps1

# macOS or Linux
source .venv/bin/activate

cd backend
python --version
python manage.py check
```

Checkpoint result

- The active branch is feature/mysql-domain-model.
- The Python interpreter comes from .venv.
- Django reports no system-check issue.

Step A2 Run the existing pages before changing the database

Terminal 1

```
python manage.py runserver
```

Open the Home, About, Contact and Workflow pages created in Lab Session 2. Record their status before continuing. Stop the server with Ctrl+C.

Page	Expected result	Observed result
/	200	
/about/	200	
/contact/	200	
/workflow/	200	

Step A3 Confirm the current SQLite configuration

Terminal

```
python manage.py shell -c "from django.conf import settings; print(settings.DATABASES['default']['ENGINE']);
print(settings.DATABASES['default']['NAME'])"
```

Checkpoint result

- The engine is django.db.backends.sqlite3.
- The name points to db.sqlite3.

Part B Install and Verify MySQL

Option 1 Ubuntu or Debian Laboratory Computer

Terminal

```
sudo apt update
sudo apt install mysql-server default-libmysqlclient-dev build-essential pkg-config
sudo systemctl enable --now mysql
sudo systemctl status mysql --no-pager
mysql --version
```

Option 2 Windows Laboratory Computer

Install MySQL Community Server 8 and MySQL Workbench from the official MySQL installer. During setup, select Server only or Developer Default, configure a strong root password, keep TCP port 3306 unless the laboratory requires another port, and start the MySQL80 Windows service. Add the MySQL bin directory to PATH if mysql is not recognised.

Windows terminal

```
mysql --version
sc query MySQL80
```

Verify the Database Service

MySQL login

```
# Ubuntu when root uses socket authentication
sudo mysql

# Windows or installations using password authentication
mysql -u root -p
```

MySQL prompt

```
SELECT VERSION();
SHOW VARIABLES LIKE 'character_set_server';
SHOW VARIABLES LIKE 'port';
EXIT;
```

Checkpoint result

- MySQL returns an 8.x server version.
- The server port is known.
- The student can enter and exit the MySQL client.

MySQL Installation Troubleshooting

Symptom	Likely cause	Correction
mysql is not recognised	Client bin directory is not on PATH	Use the full mysql path or add its bin directory to PATH.
Cannot connect through socket	MySQL service is stopped	Start mysql or MySQL80 and check its status.
Access denied for root	Wrong authentication method or password	Use sudo mysql on socket-auth Ubuntu; use the configured password elsewhere.
mysqlclient build fails	Development headers or compiler missing	Install default-libmysqlclient-dev, build-essential and pkg-config.

Part C Create the Database and Application User

Step C1 Enter MySQL as an administrator

Terminal

```
sudo mysql
# or
mysql -u root -p
```

Step C2 Create the database with full Unicode support

MySQL prompt

```
CREATE DATABASE sen305_servicedesk
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
```

Step C3 Create a dedicated application account

Use a unique local password. The sample password below is for an isolated laboratory computer and must be replaced. The Django application must not connect as MySQL root.

MySQL prompt

```
CREATE USER 'sen305_app'@'localhost'
    IDENTIFIED BY 'P@55w0rd!';

GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, INDEX,
      REFERENCES, DROP
ON sen305_servicedesk.*
TO 'sen305_app'@'localhost';

FLUSH PRIVILEGES;
```

Step C4 Verify the database, user and grants

Terminal and MySQL prompt

```
SHOW DATABASES LIKE 'sen305_servicedesk';
SHOW GRANTS FOR 'sen305_app'@'localhost';
EXIT;

mysql -u sen305_app -p sen305_servicedesk
```

MySQL prompt as sen305_app

```
SELECT DATABASE();
SHOW TABLES;
EXIT;
```

Checkpoint result

- The selected database is sen305_servicedesk.
- SHOW TABLES returns an empty result because migrations have not run.
- The application user can access only the intended database.

Record the Local Connection Without Exposing the Password

Setting	Value
Database name	sen305_servicedesk

Database user	sen305_app
Host	127.0.0.1
Port	3306 or the verified local port
Password	Stored in backend/.env; do not write it in the submitted report

Part D Configure Django for MySQL

Step D1 Install the driver and environment reader

From backend directory

```
python -m pip install mysqlclient django-environ
python -m pip freeze > requirements.txt
```

Step D2 Protect local environment files

Confirm that the repository-root .gitignore contains the following rules before creating .env.

Repository root .gitignore

```
# Environment and secrets
.env
.env.*
!.env.example

# Local database
db.sqlite3
```

Step D3 Create backend .env

backend/.env local file not committed

```
DEBUG=True
SECRET_KEY=replace-with-a-long-local-development-key
DB_NAME=sen305_servicedesk
DB_USER=sen305_app
DB_PASSWORD= P@55w0rd!
DB_HOST=127.0.0.1
DB_PORT=3306
```

Step D4 Create a safe example file

backend/.env.example committed

```
DEBUG=True
SECRET_KEY=replace-me
DB_NAME=sen305_servicedesk
DB_USER=sen305_app
DB_PASSWORD=replace-me
DB_HOST=127.0.0.1
DB_PORT=3306
```

Step D5 Load environment values in settings

Top of backend/config/settings.py

```
from pathlib import Path

import environ


BASE_DIR = Path(__file__).resolve().parent.parent

env = environ.Env(DEBUG=(bool, True))
environ.Env.read_env(BASE_DIR / ".env")

SECRET_KEY = env("SECRET_KEY")
DEBUG = env.bool("DEBUG")
```

Replace DATABASES in settings.py

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.mysql",
        "NAME": env("DB_NAME"),
        "USER": env("DB_USER"),
        "PASSWORD": env("DB_PASSWORD"),
        "HOST": env("DB_HOST", default="127.0.0.1"),
        "PORT": env("DB_PORT", default="3306"),
        "OPTIONS": {
            "charset": "utf8mb4",
        },
    }
}
```

Step D6 Verify the connection before migrating

Terminal

```
python manage.py check
python manage.py shell -c "from django.db import connection; connection.ensure_connection();
print(connection.vendor, connection.settings_dict['NAME'])"
```

Checkpoint result

- The output includes mysql sen305_servicedesk.
- No password appears in terminal output or Git.

Database Configuration Failures

Error	Meaning	Action
No module named MySQLdb	mysqlclient is not installed in active .venv	Activate .venv and reinstall mysqlclient.
Access denied for user	Credentials, host entry or grants do not match	Test the same account with mysql -u sen305_app -p.
Unknown database	DB_NAME differs from created database	Compare .env with SHOW DATABASES.
Can't connect on 127.0.0.1	Service stopped or port differs	Check service status and verified MySQL port.

Part E Recreate the Framework Tables in MySQL

Step E1 Inspect unapplied migrations

Terminal

```
python manage.py showmigrations
```

The migration files already exist in Django packages, but the new MySQL database has no migration history. Therefore, the entries should appear unchecked.

Step E2 Preview the migration plan

Terminal

```
python manage.py migrate --plan
```

Step E3 Apply the built-in migrations

Terminal

```
python manage.py migrate
```

Step E4 Verify from Django and MySQL

Terminal

```
python manage.py showmigrations
mysql -u sen305_app -p sen305_servicedesk
```

MySQL prompt

```
SHOW TABLES;
SELECT app, name, applied
FROM django_migrations
ORDER BY applied DESC
LIMIT 10;
EXIT;
```

Checkpoint result

- auth, admin, contenttypes and sessions migrations are checked.
- MySQL contains django_migrations and other framework tables.
- The original public pages still return 200 after runserver starts.

Important Practical Observation

Changing the configured database did not convert db.sqlite3. Django replayed the same migration history into an empty MySQL database. This is why migration files belong in version control and database files do not.

Part F Design the Service Request Domain

Step F1 Create the domain application

Terminal

```
python manage.py startapp requests_app
```

Step F2 Register the application

backend/config/settings.py relevant entries

```
INSTALLED_APPS = [
    # Django applications...
    "core.apps.CoreConfig",
    "requests_app.apps.RequestsAppConfig",
]
```

Step F3 Draw the entity relationships before coding

Parent	Relationship	Child	Deletion rule
User	one to many	ServiceRequest as requester	Protect request history
Category	one to many	ServiceRequest	Protect categories in use
ServiceRequest	one to zero or one	WorkOrder	Cascade with request
User	one to many	WorkOrder as technician	Protect assignment history
ServiceRequest	one to many	Comment	Cascade with request
User	one to many	Comment as author	Protect comment authorship

Step F4 Prepare the application file layout

Expected layout

```
requests_app/
├── migrations/
│   └── __init__.py
├── __init__.py
├── admin.py
├── apps.py
├── models.py
└── tests.py
```

Design Worksheet

Business question	Chosen implementation
How is a request identified externally?	UUID primary key
Can two categories have the same name?	No; Category.name is unique
Valid priority range	1 highest to 5 lowest
Initial request status	open
Can one request have several work orders in this	No; OneToOneField

lab?	
What happens when an in-use category is deleted?	Deletion is blocked by PROTECT
How are default lists ordered?	Newest requests and comments first

Part G Implement the Django Models

Step G1 Replace requests_app models with the domain model

backend/requests_app/models.py

```
import uuid

from django.conf import settings
from django.core.validators import MaxValueValidator, MinValueValidator
from django.db import models


class Category(models.Model):
    name = models.CharField(max_length=80, unique=True)
    description = models.TextField(blank=True)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ("name",)
        verbose_name_plural = "categories"

    def __str__(self):
        return self.name


class ServiceRequest(models.Model):
    class Status(models.TextChoices):
        OPEN = "open", "Open"
        ASSIGNED = "assigned", "Assigned"
        IN_PROGRESS = "in_progress", "In progress"
        RESOLVED = "resolved", "Resolved"
        CLOSED = "closed", "Closed"

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    requester = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.PROTECT,
        related_name="service_requests",
    )
    category = models.ForeignKey(
        Category,
        on_delete=models.PROTECT,
        related_name="service_requests",
    )
    title = models.CharField(max_length=160)
    description = models.TextField()
    location = models.CharField(max_length=160)
    priority = models.PositiveSmallIntegerField(
        default=3,
        validators=[MinValueValidator(1), MaxValueValidator(5)],
    )
    status = models.CharField(
        max_length=20,
        choices=Status.choices,
        default=Status.OPEN,
        db_index=True,
    )
    resolved_at = models.DateTimeField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ("-created_at",)
        indexes = [
            models.Index(fields=("category", "status"), name="req_cat_status_idx"),
            models.Index(fields=("priority", "created_at"), name="req_priority_date_idx"),
        ]
        constraints = [
            models.CheckConstraint(
                condition=models.Q(priority__gte=1, priority__lte=5),
```

```
                name="request_priority_between_1_and_5",
            ),
        ]

    def __str__(self):
        return f"{self.title} ({self.get_status_display()})"


class WorkOrder(models.Model):
    service_request = models.OneToOneField(
        ServiceRequest,
        on_delete=models.CASCADE,
        related_name="work_order",
    )
    technician = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.PROTECT,
        related_name="assigned_work_orders",
    )
    work_notes = models.TextField(blank=True)
    started_at = models.DateTimeField(null=True, blank=True)
    completed_at = models.DateTimeField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ("-created_at",)

    def __str__(self):
        return f"Work order for {self.service_request.title}"


class Comment(models.Model):
    service_request = models.ForeignKey(
        ServiceRequest,
        on_delete=models.CASCADE,
        related_name="comments",
    )
    author = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.PROTECT,
        related_name="service_request_comments",
    )
    body = models.TextField()
    is_internal = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ("-created_at",)

    def __str__(self):
        return f"Comment by {self.author} on {self.service_request.title}"
```

Step G2 Run validation without creating a migration

Terminal

```
python manage.py check
```

Checkpoint result

- The models import correctly.
- Index and constraint names are accepted.
- Reverse relation names do not clash.

Model Field Inspection Task

Field choice	Practical reason
UUIDField primary key	Avoids predictable sequential public identifiers

PROTECT on requester and category	Preserves historical request references
CASCADE on comments and work order	Dependent records disappear with their owning request
TextChoices for status	Provides controlled stored values and readable labels
Validators plus DB check	Gives application feedback and database enforcement
db_index on status	Supports frequent workflow filtering
Composite indexes	Support common category/status and priority/date queries

Part H Generate and Inspect the Migration

Step H1 Create the initial migration

Terminal

```
python manage.py makemigrations requests_app
```

Step H2 Open and review the generated file

Open requests_app/migrations/0001_initial.py. Locate CreateModel operations, foreign keys, options, indexes and the priority check constraint. Do not edit the generated file merely to make it shorter.

Migration element	Student evidence
Category CreateModel	Line or screenshot: _____
ServiceRequest UUID primary key	Line or screenshot: _____
Foreign keys and deletion rules	Line or screenshot: _____
req_cat_status_idx	Line or screenshot: _____
request_priority_between_1_and_5	Line or screenshot: _____

Step H3 Preview the SQL Django will execute

Terminal

```
python manage.py sqlmigrate requests_app 0001
```

Find the CREATE TABLE statements, foreign-key constraints, CHECK constraint and CREATE INDEX statements. The precise SQL syntax is generated for MySQL, not copied from SQLite.

Step H4 Preview and apply the migration

Terminal

```
python manage.py migrate --plan
python manage.py migrate requests_app
python manage.py showmigrations requests_app
```

Step H5 Verify the schema in MySQL

Terminal

```
mysql -u sen305_app -p sen305_servicedesk
```

MySQL prompt

```
SHOW TABLES LIKE 'requests_app%';
DESCRIBE requests_app_servicerequest;
SHOW CREATE TABLE requests_app_servicerequest\G
SHOW INDEX FROM requests_app_servicerequest;
EXIT;
```

Checkpoint result

- 0001_initial is marked applied.
- Four application tables exist.
- The priority check and both named indexes are visible.

Part I Configure Django Admin

Step I1 Register the models with useful list displays

backend/requests_app/admin.py

```
from django.contrib import admin

from .models import Category, Comment, ServiceRequest, WorkOrder


@admin.register(Category)
class CategoryAdmin(admin.ModelAdmin):
    list_display = ("name", "is_active", "created_at")
    list_filter = ("is_active",)
    search_fields = ("name", "description")


class CommentInline(admin.TabularInline):
    model = Comment
    extra = 0
    fields = ("author", "body", "is_internal", "created_at")
    readonly_fields = ("created_at",)


@admin.register(ServiceRequest)
class ServiceRequestAdmin(admin.ModelAdmin):
    list_display = (
        "title", "category", "requester", "priority", "status", "created_at",
    )
    list_filter = ("status", "priority", "category")
    search_fields = ("title", "description", "location", "requester__username")
    readonly_fields = ("id", "created_at", "updated_at")
    autocomplete_fields = ("requester", "category")
    date_hierarchy = "created_at"
    inlines = (CommentInline,)


@admin.register(WorkOrder)
class WorkOrderAdmin(admin.ModelAdmin):
    list_display = ("service_request", "technician", "started_at", "completed_at")
    search_fields = ("service_request__title", "technician__username")
    autocomplete_fields = ("service_request", "technician")


@admin.register(Comment)
class CommentAdmin(admin.ModelAdmin):
    list_display = ("service_request", "author", "is_internal", "created_at")
    list_filter = ("is_internal", "created_at")
    search_fields = ("body", "service_request__title", "author__username")
```

Step I2 Create the administrator

Terminal

```
python manage.py createsuperuser
```

Use a laboratory-only account and a strong password. Do not write the password in the report or commit it anywhere.

Step I3 Start the server and enter Admin

Terminal 1

```
python manage.py runserver
```

Open http://127.0.0.1:8000/admin/. Log in and confirm that Categories, Service requests, Work orders and Comments appear under Requests App.

Step I4 Create controlled demonstration records

Order	Record	Required values
1	Category	Electrical; active
2	Category	ICT Support; active
3	User	requester1
4	User	technician1
5	Service request	Broken laboratory socket; Electrical; priority 2; requester1
6	Work order	Assign the request to technician1
7	Comment	Initial inspection scheduled; authored by technician1

Checkpoint result

- List filters change the visible service requests.
- Search finds the request by title or location.
- Related records display readable __str__ values instead of object numbers.

Part J Perform ORM Operations in Django Shell

Step J1 Open the enhanced Django shell

Terminal 2

```
python manage.py shell
```

Step J2 Import models and create users if they do not exist

Django shell

```
from django.contrib.auth import get_user_model
from django.db.models import Count, Q
from requests_app.models import Category, Comment, ServiceRequest, WorkOrder

User = get_user_model()
requester, _ = User.objects.get_or_create(username="requester1")
technician, _ = User.objects.get_or_create(username="technician1")
```

Step J3 Create or retrieve categories

Django shell

```
electrical, _ = Category.objects.get_or_create(
    name="Electrical",
    defaults={"description": "Electrical faults and maintenance"},
)
ict, _ = Category.objects.get_or_create(
    name="ICT Support",
    defaults={"description": "Computers, networks and software"},
)
Category.objects.all()
```

Step J4 Create a service request

Django shell

```
request_item = ServiceRequest.objects.create(
    requester=requester,
    category=electrical,
    title="Intermittent power in Lab 2",
    description="Sockets lose power when several systems are connected.",
    location="Software Engineering Lab 2",
    priority=2,
)
request_item.id
request_item.status
request_item.get_status_display()
```

Step J5 Create related work and comment records

Django shell

```
work_order = WorkOrder.objects.create(
    service_request=request_item,
    technician=technician,
    work_notes="Inspect distribution board and socket circuit.",
)

Comment.objects.create(
    service_request=request_item,
    author=technician,
    body="Inspection scheduled for the next laboratory period.",
    is_internal=False,
)
```

```
request_item.work_order.technician.username
request_item.comments.count()
```

Part K Query Update and Aggregate the Data

Step K1 Filter requests by workflow state and priority

Django shell

```
ServiceRequest.objects.filter(status="open")
ServiceRequest.objects.filter(priority__lte=2)
ServiceRequest.objects.filter(category__name="Electrical")
ServiceRequest.objects.filter(
    Q(title__icontains="power") | Q(description__icontains="power")
)
```

Step K2 Traverse relationships in both directions

Django shell

```
requester.service_requests.all()
electrical.service_requests.all()
technician.assigned_work_orders.all()
request_item.comments.select_related("author").all()
request_item.work_order.technician
```

Step K3 Update a workflow record

Django shell

```
request_item.status = ServiceRequest.Status.ASSIGNED
request_item.save(update_fields=["status", "updated_at"])
request_item.refresh_from_db()
request_item.status
```

Step K4 Aggregate by category and status

Django shell

```
Category.objects.annotate(
    request_count=Count("service_requests")
).values("name", "request_count")

ServiceRequest.objects.values("status").annotate(
    total=Count("id")
).order_by("status")
```

Step K5 Inspect generated SQL

Django shell

```
queryset = ServiceRequest.objects.select_related(
    "requester", "category"
).filter(status="assigned")
print(queryset.query)
queryset.explain()
```

ORM Evidence Worksheet

Operation	Observed result
Count of all service requests	_____
Count of open requests	_____
Count of assigned requests	_____

Electrical category request count	_____
Technician work-order count	_____
Index or access method shown by explain	_____

Part L Prove Data Integrity

Test 1 Duplicate Category Name

Django shell

```
from django.db import IntegrityError, transaction

try:
    with transaction.atomic():
        Category.objects.create(name="Electrical")
except IntegrityError as error:
    print(type(error).__name__, error)
```

Checkpoint result

- The database rejects the duplicate unique name.
- The surrounding atomic block prevents a broken transaction state.

Test 2 Priority Outside the Database Range

Django shell

```
from django.db import IntegrityError, transaction

try:
    with transaction.atomic():
        ServiceRequest.objects.create(
            requester=requester,
            category=electrical,
            title="Invalid priority test",
            description="This row must not persist.",
            location="Test location",
            priority=9,
        )
except IntegrityError as error:
    print(type(error).__name__, error)
```

Model validators run through forms and full_clean(), while the database CHECK constraint protects direct ORM writes. Verify that no Invalid priority test row exists.

Django shell

```
ServiceRequest.objects.filter(title="Invalid priority test").exists()
```

Test 3 Application Level Validation

Django shell

```
invalid = ServiceRequest(
    requester=requester,
    category=electrical,
    title="Validator test",
    description="Test validation before saving.",
    location="Test location",
    priority=0,
)
invalid.full_clean()
```

Checkpoint result

- ValidationError identifies priority.
- The invalid instance is not saved.

Test 4 Protected Category Deletion

Django shell

```
from django.db.models import ProtectedError

try:
    electrical.delete()
except ProtectedError as error:
    print(type(error).__name__, len(error.protected_objects))
```

Checkpoint result

- Deletion is blocked while service requests reference Electrical.
- Historical records remain intact.

Test 5 One Work Order per Request

Django shell

```
try:
    with transaction.atomic():
        WorkOrder.objects.create(
            service_request=request_item,
            technician=technician,
        )
except IntegrityError as error:
    print(type(error).__name__, error)
```

Checkpoint result

- The unique one-to-one relation rejects the second work order.

Part M Write Automated Model Tests

Step M1 Create model tests

backend/requests_app/tests.py

```
from django.contrib.auth import get_user_model
from django.core.exceptions import ValidationError
from django.db import IntegrityError, transaction
from django.db.models import ProtectedError
from django.test import TestCase

from .models import Category, ServiceRequest, WorkOrder


class ServiceRequestModelTests(TestCase):
    def setUp(self):
        user_model = get_user_model()
        self.requester = user_model.objects.create_user(username="requester")
        self.technician = user_model.objects.create_user(username="technician")
        self.category = Category.objects.create(name="Electrical")

    def make_request(self, **overrides):
        values = {
            "requester": self.requester,
            "category": self.category,
            "title": "Faulty socket",
            "description": "Socket does not supply power.",
            "location": "Software Engineering Lab",
            "priority": 2,
        }
        values.update(overrides)
        return ServiceRequest.objects.create(**values)

    def test_default_status_is_open(self):
        item = self.make_request()
        self.assertEqual(item.status, ServiceRequest.Status.OPEN)

    def test_priority_validator_rejects_zero(self):
        item = ServiceRequest(
            requester=self.requester,
            category=self.category,
            title="Invalid",
            description="Invalid priority.",
            location="Lab",
            priority=0,
        )
        with self.assertRaises(ValidationError):
            item.full_clean()

    def test_database_constraint_rejects_priority_six(self):
        with self.assertRaises(IntegrityError):
            with transaction.atomic():
                self.make_request(priority=6)

    def test_category_in_use_is_protected(self):
        self.make_request()
        with self.assertRaises(ProtectedError):
            self.category.delete()

    def test_request_accepts_only_one_work_order(self):
        item = self.make_request()
        WorkOrder.objects.create(
            service_request=item,
            technician=self.technician,
        )
        with self.assertRaises(IntegrityError):
            with transaction.atomic():
                WorkOrder.objects.create(
                    service_request=item,
                    technician=self.technician,
                )
```

Step M2 Run the application tests

Terminal

```
python manage.py test requests_app -v 2
```

Step M3 Run final framework checks

Terminal

```
python manage.py check
python manage.py makemigrations --check --dry-run
```

Checkpoint result

- All five model tests pass.
- System check reports no issues.
- No uncommitted model change requires another migration.

Part N Complete the In Lab Exercise

Complete the following extension without changing the existing migration file. Change models.py, generate a new 0002 migration, inspect it, apply it, update Admin, and add tests.

Task 1 Add Service Level Due Date

1. Add due_at as a nullable DateTimeField to ServiceRequest.
2. Add an index that supports status and due_at filtering.
3. Create and inspect migration 0002.
4. In Admin, display due_at and add an overdue filter strategy.
5. Create two requests with different due dates and query those due before a chosen time.

Task 2 Add Resolution Summary

6. Add resolution_summary as a blank TextField.
7. Create model validation so RESOLVED or CLOSED requests require resolved_at and a non-empty resolution summary.
8. Prove the rule with full_clean() and automated tests.
9. Do not place the rule only in Django Admin; it belongs to the domain model or service layer.

Task 3 Analyse a Query

Create at least ten synthetic service requests across two categories and three statuses. Run a query for open or assigned requests in one category ordered by priority and created date. Print queryset.query and queryset.explain(). Record which index is used or explain why the dataset is too small for MySQL to prefer an index.

Exercise Acceptance Criteria

Criterion	Pass condition
Migration discipline	A reviewed 0002 migration applies successfully
Due date	Field and supporting index exist in MySQL
Resolution rule	Invalid resolved records fail full_clean()
Admin	Due date and resolution data are visible and searchable or filterable
Query evidence	Generated SQL and explain output are recorded
Tests	Original and new tests pass
Data safety	Only synthetic laboratory data are used

Part O Version Control and Submission

Step O1 Review secrets and generated files

From repository root

```
git status
git check-ignore backend/.env backend/db.sqlite3
git diff -- .gitignore backend/.env.example backend/config backend/requests_app backend/requirements.txt
```

backend/.env must be ignored. Migration files must be visible to Git. Do not submit a MySQL data directory or database export containing credentials.

Step O2 Run final verification

Terminal

```
cd backend
python manage.py check
python manage.py showmigrations requests_app
python manage.py test requests_app -v 2
python manage.py makemigrations --check --dry-run
```

Step O3 Commit the increment

Terminal

```
cd ..
git add .gitignore backend/.env.example backend/config backend/requests_app backend/requirements.txt
git status
git commit -m "feat: add MySQL service request domain model"
git log -1 --oneline
git push -u origin feature/mysql-domain-model
```

Required Submission Evidence

Evidence	What to submit or demonstrate
ERD	Four models with cardinalities and deletion rules
Database connection	Django reports mysql and sen305_servicedesk without exposing password
Migrations	0001 and exercise 0002 plus sqlmigrate evidence
Schema	SHOW CREATE TABLE and SHOW INDEX evidence
Admin	Related records, filters and searches
ORM	Create, relationship, filter, update, aggregate and explain outputs
Integrity	Duplicate, priority, PROTECT and one-to-one failures
Tests	Passing test output
Repository	Feature branch URL and final commit hash

Quick Troubleshooting Reference

Error or symptom	Most likely cause	First corrective action
No module named MySQLdb	mysqlclient missing from active environment	Activate .venv and reinstall driver dependencies.
Access denied	Credentials or grants differ	Test the same user through the mysql client.
Unknown database	Wrong DB_NAME or database not created	Compare .env with SHOW DATABASES.
No changes detected	Application not registered or file not saved	Check INSTALLED_APPS and save models.py.
Table does not exist	Migration not applied to selected database	Check connection vendor, showmigrations and migrate.
Duplicate key	Unique field already has that value	Inspect existing row and use get_or_create where appropriate.
ProtectedError	A protected related row exists	Preserve history or reassign references; do not force deletion.
TransactionManagementError	IntegrityError left transaction broken	Wrap the expected failure in transaction.atomic().
Admin autocomplete error	Related Admin lacks search_fields	Add search_fields to the related ModelAdmin.
Migration history conflict	Files and database history disagree	Stop; inspect dependencies and applied migrations before changing anything.

End of Session Checklist

- MySQL service is running and accessible to sen305_app.
- Django uses MySQL rather than SQLite.
- backend/.env is ignored and .env.example contains no secret.
- Category, ServiceRequest, WorkOrder and Comment tables exist.
- Relationships, indexes and the priority constraint are visible.
- Django Admin can create and search related records.
- Required ORM queries and aggregates work.
- Invalid values and protected deletions are rejected.
- All model tests pass.
- Migrations are committed and the database itself is not committed.